COMPLETE BUILD ROADMAP • HINDI

मनीषा AI Agent

3D अवतार, वॉइस, कैमरा, मोबाइल टास्क और स्मार्ट ऑटोमेशन वाला **Android-first Flutter Assistant**

Flutter + Kotlin | VS Code + Codex | 3D Avatar + Pet | AI Model Hub | Mission Control

> **यह गाइड किसके लिए है?**
> Acer Nitro Lite 16 लैपटॉप • 16 GB RAM • 512 GB SSD (लगभग 400 GB खाली) • Windows पर नए रीसेट के बाद शुरुआत।
> **निर्णय:** अभी Cursor का paid plan लेने की आवश्यकता नहीं; पहले मुफ्त VS Code में अपने ChatGPT Plus के साथ Codex इस्तेमाल करें।

एक दिन में लक्ष्य: **सही development setup + असली Android फोन पर चलने वाला पहला मनीषा Flutter demo**। पूरे Jarvis-जैसे सिस्टम को कई चरणों में भरोसेमंद बनाया जाएगा।

तैयार: 9 नवंबर 2026 • निजी प्रोजेक्ट ब्लूप्रिंट • सार्वजनिक रिलीज़ से पहले ऐप-स्टोर नीतियाँ पुनः जाँचें।

01. पहले यह पढ़ो: एक दिन में क्या पूरा होगा?

मनीषा की पूरी कल्पना बड़ी है। **पहले दिन पूरा प्रोडक्शन ऐप बनना यथार्थवादी नहीं है।** पहली सफलता होगी: विकास टूल चलें, फोन कनेक्ट हो, और स्क्रीन पर “Manisha” वाला Flutter ऐप दिखे। तभी अगला AI फीचर जोड़ना आसान होगा।

> **पहले दिन का Done मतलब:** `flutter doctor` में आवश्यक Android सेटअप ठीक; `flutter devices` में फोन दिखे; `flutter run` से ऐप खुले; Git में पहला commit बने।

क्रम	आज/अगले काम वाले दिन क्या करना है	कैसे जाँचें
1	Git, VS Code, Android Studio इंस्टॉल	सभी ऐप लॉन्च हो जाएँ
2	VS Code से Flutter SDK, Flutter/Dart extension	`flutter --version`
3	Android SDK और licenses	`flutter doctor -v`
4	USB debugging वाला असली फोन कनेक्ट	`flutter devices`
5	पहला ऐप बनाकर चलाना; Codex से UI सुधरवाना	`flutter run`
6	Git save / अगले चरण का task list	`git status`

> **PC की क्षमता:** 16 GB RAM और खाली SSD शुरुआती Flutter + Android Studio + Blender के लिए ठीक है। एक ही समय में बड़े emulator, Blender render और local AI model न चलाएँ। शुरुआती परीक्षण असली Android फोन पर करें। GPU का मॉडल अभी ज्ञात नहीं, इसलिए भारी 3D rendering की क्षमता का दावा नहीं।

02. PC में क्या-क्या डाउनलोड करना है

सॉफ़्टवेयर	क्यों चाहिए	डाउनलोड
Git	कोड का history / rollback	git-scm.com/downloads
VS Code	मुफ्त editor, Codex और Flutter extensions	code.visualstudio.com/Download
Flutter SDK	Flutter ऐप बनाने की मुख्य toolkit; VS Code से डाउनलोड करें	docs.flutter.dev/install/with-vs-code
Android Studio	Android SDK, Platform Tools, build tools	developer.android.com/studio
Codex extension	ChatGPT अकाउंट से coding agent	OpenAI Codex • VS Code Marketplace
Blender	3D अवतार और पेट, rig/animation	blender.org/download
Python (बाद में)	जरूरत पड़ने पर orchestration backend / tests	python.org/downloads

अभी मत खरीदो: Cursor Pro आवश्यक नहीं। बाद में UI/वर्कफ़्लो बेहतर लगे तभी तुलना करके लो। **अभी मत इंस्टॉल करो:** Unity, भारी local LLM, कई emulator, बेकार plugins—पहला Flutter ऐप बनने तक इनकी आवश्यकता नहीं।

इंस्टॉलेशन का सही क्रम

1. Windows Update पूरा करें, फिर **Git** और **VS Code** इंस्टॉल करें।
2. VS Code → Extensions (Ctrl+Shift+X) → **Flutter** खोजकर इंस्टॉल करें। Dart साथ आता है।
3. Ctrl+Shift+P → **Flutter: New Project** → SDK न मिले तो **Download SDK** → उदाहरण के लिए `C:\src` चुनें → Clone Flutter। निर्देश आएँ तो PATH में जोड़ें, नया टर्मिनल खोलें।
4. Android Studio खोलें → Setup Wizard से **Android SDK, Platform Tools, Command-line Tools** इंस्टॉल/जाँचें। SDK Manager में missing tools जोड़ें।
5. टर्मिनल में नीचे दी गई doctor commands चलाएँ। Flutter डॉक्टर जो गलती बताए, सिर्फ वही सुधारें।
6. VS Code में **OpenAI Codex** extension इंस्टॉल करें → **Sign in with ChatGPT** → Plus अकाउंट से लॉगिन करें।
7. Blender बाद में इंस्टॉल करें, जब Flutter demo चल जाए।

03. पहला Flutter ऐप फोन पर चलाना — कॉपी करने वाली कमांड

A. Setup check (PowerShell / VS Code Terminal)

```
git --version
flutter --version
flutter doctor -v
flutter doctor --android-licenses
```

Licenses पर शर्तें पढ़कर **y** चुनें। `flutter doctor` में Windows desktop / Visual Studio से जुड़ी warning आ सकती है; सिर्फ Android ऐप बना रहे हैं तो Android वाली errors प्राथमिक हैं।

B. Android फोन की तैयारी

1. फोन में Settings → About phone → Build number पर कई बार टैप करके **Developer Options** चालू करें (फोन के अनुसार नाम अलग हो सकता है)।
2. Developer Options में **USB debugging** चालू करें।
3. डेटा-सपोर्टेड USB केबल से लैपटॉप में फोन जोड़ें। फोन पर आए **Allow USB debugging** संवाद को अपने भरोसेमंद PC के लिए स्वीकार करें।
4. टर्मिनल में `flutter devices` चलाएँ। फोन न दिखे तो cable, USB mode, Windows driver और debugging authorization जाँचें।

C. Project और पहला run

```
cd $HOME
mkdir Projects
cd Projects
flutter create manisha
cd manisha
flutter devices
flutter run
```

ये PowerShell commands हैं। `Projects` फ़ोल्डर पहले से बना हो तो `mkdir Projects` छोड़ दें।

सफलता की जाँच: फोन पर Flutter का default counter ऐप खुल जाए तो setup सही है। अब `lib/main.dart` को Codex से मनीषा के होम स्क्रीन में बदलवाएँ।

D. पहला सुरक्षित checkpoint

```
git init
git add .
git commit -m "Initial Manisha Flutter app"
```

अगर Git पहचान माँगे तो एक बार `git config --global user.name "Your Name"` और `git config --global user.email "you@example.com"` सेट करें। Example email को अपना मान्य email दें; API keys या निजी फाइलें कभी Git में commit न करें।

04. Codex को क्या बोलना है? तैयार Prompts

Codex को एक बार में “पूरा Jarvis बना दो” मत कहें। छोटे काम दें और हर बदलाव के बाद ऐप चलाकर देखें। ये prompts सीधे कॉपी कर सकते हैं:

Prompt 1 — पहले दिन का UI (सिर्फ mock data)

```
Create a production-style Flutter Android app named Manisha. Keep code modular and
compilable. Build a polished Home screen with a 3D-avatar placeholder, Hindi/English chat
cards, microphone button (UI only), task status card, and bottom tabs: Home, Vision, Tasks,
Models, Settings. Use mock data only. Do not add API keys or request sensitive permissions.
Run flutter analyze and fix errors. Explain changed files and how to test on a phone.
```

Prompt 2 — Settings + Model Hub

```
Add a Model Hub screen to the existing Flutter app. Users can create provider profiles with display name, model ID, API base URL (only supported providers), local credential reference, priority, enabled flag and budget. Add model auto-failover settings and an optional combo profile. Do not hardcode secrets, log API keys or perform real paid requests. Use mocked adapters first, add tests, and preserve the existing UI.
```

Prompt 3 — Tasks + Vision Dashboard

```
Build Tasks and Vision Dashboard screens with local mock data. A task has title, schedule, status, permission state, confirmation requirement, last result and activity log. Display upcoming, running, needs approval, completed and failed. Provide an always-visible Stop button and accessible Hindi labels. Do not automate third-party apps yet. Add widget tests.
```

Prompt 4 — Camera + avatar preparation

```
Prepare an optional camera preview feature with explicit user consent and visible camera state. Keep it disabled by default. Add an avatar interface that supports emotion, lip-sync amplitude and animation state via a mock controller. Do not store camera frames or face biometrics. Explain the permissions, Android lifecycle and how users can revoke access.
```

> **Codex workflow:** पहले plan दिखाने को कहो → बदलाव approve/review करो → `flutter analyze` और फोन test → Git commit। Agent को हर command / permission बिना देखे approve मत करना।

05. मनीषा में आखिर क्या-क्या होगा? पूरा फीचर मैप

मॉड्यूल	फीचर्स	कब
Conversation	हिंदी/English chat, voice talk, subtitles, interrupt, voice selection, 10 भाषाओं के विकल्प (मॉडल की क्षमता के अनुसार)	पहले से मध्य
3D Companion	3D लड़की का अवतार, आँख झपकना, lip-sync, gestures, emotions; cute original 3D pet	मध्य
Phone Tools	ऐप खोलना, supported deep link, reminders, notes, सीमित और अनुमत device actions	मध्य
Smart Planner	एक बार/दोहराए जाने वाले scheduled tasks, reminders, task status, retry, cancel	मध्य
Model Hub	कई providers, custom model ID, BYOK, fallback order, Combo, Auto Switch, cost caps	पहले से मध्य
Vision	Camera preview और scene understanding (opt-in), activity feed, क्यों कोई task रुका	मध्य
Identity	Owner / Guest, optional speaker identification, optional face matching, OS biometrics for sensitive actions	बाद में

Mission Control	कामों का dashboard, running/queued/failed/needs approval, audit trail, Emergency Stop	पहले से मध्य
Privacy	Permission Center, delete data, export data, voice/mic/camera indicators, retention controls	शुरुआत से
Integrations	MCP tool connectors, smart-home integration, files, calendar, permissions/ approval per tool	बाद में

सुझाई गई 10 UI भाषाएँ

हिंदी, English, बंगाली, तमिल, तेलुगु, मराठी, गुजराती, कन्नड़, मलयालम और पंजाबी। **ध्यान:** ऐप की लिखित UI भाषा और speech-to-text / text-to-speech की वास्तविक भाषा-सपोर्ट अलग चीज़ें हैं; हर provider का समर्थन जाँचना होगा।

06. ऐप की स्क्रीनें और यूज़र फ्लो

स्क्रीन	मुख्य काम
Welcome / Onboarding	मनीषा परिचय, भाषा चुनाव, privacy explanation
Permission Center	Camera / mic / notifications / optional special access को समझकर Allow, Not now, Settings
Home	अवतार, pet, बोलो-बटन, chat, quick actions
Vision / Mission Control	अभी क्या कर रही है, कौन-सा tool चला, result, किसे मंज़ूरी चाहिए
Tasks	नया टास्क, schedule, history, cancel, retry
Model Hub	providers, model profile, API key जोड़ना, fallback/Combo, खर्च की सीमा
Persona + Pet	आवाज़, चेहरे के भाव, pet का व्यवहार और appearances
Security & Settings	Owner/Guest, data delete/export, permission revoke, Emergency Stop

बोलकर काम करवाने का फ्लो

यूज़र की आवाज़ → speech recognition → intent + model router → **permission + action preview** → approved tool → result log → voice answer + avatar expression

उदाहरण: “मनीषा, शाम सात बजे पानी पीने की याद दिलाना।” → समय parse → notification permission की जाँच → local reminder save → Tasks में दिखाओ → 7 बजे notification। यह पहले सुरक्षित और यथार्थवादी automation उदाहरण है।

07. AI Models, BYOK और Auto-Failover

प्रोफ़ाइल: Provider name + model ID + supported endpoint + secret reference + timeout + priority + daily budget + active/disabled। यूज़र कई प्रोफ़ाइल सेव कर सकेगा। Custom model नाम लिखने से वह मॉडल अपने-आप उपलब्ध नहीं होता—उसका provider endpoint और access सही होना चाहिए।

> **Auto fallback (बिना Combo):** Primary A विफल → allowed backup B → allowed backup C। **Combo:** यूज़र अपने नाम से A+B+C का समूह सेव करे, खास priority/strategy के साथ। Auto Switch हर retry पर cost policy और provider compatibility चेक करे।

- **Free-only switch:** paid fallback बिल्कुल नहीं; quota खत्म हो तो साफ error।
- **Budget mode:** user-approved limit तक paid provider इस्तेमाल हो।
- **Streaming failover:** अधूरा जवाब दूसरे मॉडल से चुपके से जोड़ने के बजाय नई attempt के रूप में दिखाएँ।
- **Secrets:** व्यक्तिगत keys को सुरक्षित storage/Android Keystore से सुरक्षित करें; server होने पर encrypted vault और per-user isolation रखें। API key को analytics, logs या repository में न भेजें।
- **Safe endpoints:** random URL पर secret भेजने से पहले चेतावनी, known providers के लिए allowlist और स्पष्ट domain confirmation दें।

क़ीमत: ChatGPT Plus में Codex का इस्तेमाल और मनीषा में OpenAI/API providers से inference कराना अलग बिलिंग हो सकती है। OpenRouter जैसे aggregator को provider adapter की तरह जोड़ सकते हैं; किसी भी सेवा का नाम verified implementation से पहले built-in support के रूप में दावा न करें।

08. Voice, 3D Avatar, 3D Pet और Camera

Voice stack

रास्ता 1: Audio capture → STT → LLM → TTS → speaker output। रास्ता 2: किसी supported realtime voice platform से integration। दोनों में latency, भाषाओं और कीमत का टेस्ट करें। **Agora RTC** लाइव audio transport है; वह अपने-आप AI brain या TTS नहीं बन जाता। Agora के **10,000 free monthly RTC minutes** और Conversational AI के **300 free minutes** अलग quotas हैं; प्रकाशित शर्तें बदल सकती हैं।

3D अवतार कैसे बनेगा?

1. Blender में original stylized लड़की का model बनाओ; शुरुआत में low-poly ताकि फोन पर smooth रहे।
2. Face rig / blend shapes (mouth open, smile, blink), skeleton और idle/talk/listen/happy animations तैयार करो।
3. Model को `.glb` / glTF में export करो। Material, animations और runtime compatibility टेस्ट करो।
4. Flutter स्क्रीन में supported 3D renderer (जैसे native integration या WebView-based Three.js prototype) जोड़ो; model load/animation performance असली फोन पर जाँचो।
5. TTS audio level या phoneme timing के आधार पर lip sync; AI के चुने `emotion` state के आधार पर expressions बदलो।

Pet: original छोटे fox/cat/robot creature का companion, साथ चलना, blink, sleep, celebrate, idle effects। PUBG / Free Fire की कॉपी या copyrighted assets इस्तेमाल न करें।

Camera + पहचान

यूज़र जब camera screen खोले और अनुमति दे, तब सामने/पीछे के कैमरे से दृश्य समझें। Face Recognition और Voice Recognition को पहचान के संकेत की तरह लें—सुरक्षित Owner actions के लिए Android **BiometricPrompt / device credential** से पुष्टि कराएँ। चेहरे का raw डेटा बिना स्पष्ट आवश्यकता और informed consent के न रखें। Camera active indicator, stop/close button और data deletion अनिवार्य डिजाइन निर्णय हैं।

09. Android automation: क्या संभव है, क्या सीमित है

काम	व्यावहारिक तरीका	सीमा
App खोलना	Android intent / supported deep link	ऐप installed / exported activity होना चाहिए
YouTube search	Supported link/intent से YouTube में search खोलना	स्क्रीन automation हर स्थिति में भरोसेमंद नहीं
Like, comment, subscribe	यूज़र की स्पष्ट review/confirmation के साथ अनुमत flow	third-party UI बदल सकती है; platform policy लागू
Reminder/schedule	WorkManager या AlarmManager + notification	exact alarm के लिए special access/policy हो सकती है
Background voice	उपयुक्त foreground service + visible notification	Android mic/camera background-start restrictions
Screen actions	केवल narrow, disclosed, rule-based approved capabilities	Google Play सामान्य autonomous Accessibility agents प्रतिबंधित करता है

> **महत्वपूर्ण नियम:** “Allow All” से किसी ऐप को असीमित फोन नियंत्रण नहीं मिल जाता। Google Play की Accessibility policy सामान्य AI ऐप में स्वतंत्र रूप से योजना बनाकर दूसरे ऐप्स पर काम करने वाले agent की अनुमति नहीं देती। इसलिए उद्देश्य-विशिष्ट, narrow, reviewable commands, उपयोगकर्ता की मंज़ूरी और नीति-अनुकूल integrations चुनें।

Permission Center — user experience

पहले एक स्क्रीन पर features समझाओ; “Continue”, “Not now” और “Manage later” दो। Android की real system permission prompts **एक-एक करके, ज़रूरत के समय** दिखाओ। Accessibility और exact alarms जैसे special access साधारण Allow/Don't Allow permission dialog नहीं हैं। Permission इंकार होने पर सीमित functionality के साथ ऐप चलता रहे। ऐप में कभी भी permission बंद करने का रास्ता हो।

Emergency Stop + logs

सभी pending actions cancel, active microphones/camera बंद, background sessions end, और last action log—ये controls हमेशा आसानी से उपलब्ध हों। संवेदनशील काम (मैसेज भेजना, भुगतान, पोस्ट करना, delete करना, unsubscribe करना) पहले review/approval माँगें।

10. MCP, Plugins और Architecture

Flutter App (UI, Avatar, Tasks, Permissions)

↓

Agent Orchestrator (intent, policies, approvals)

↓

Model Router (provider + BYOK + fallback) | Tool Registry / MCP connectors

↓

Android Kotlin Bridge / Local DB / Approved Backend

↓

Vision Dashboard + audit logs + result voice

Flutter: interface, navigation, animations, task display। **Kotlin native bridge:** Android intents, notification/foreground service, secure credential handling, approved system integrations। **Orchestrator:** requests को छोटे steps में बाँटे, permissions चेक करे और सुरक्षित actions का result ले। **MCP:** compatible tools के लिए standard interface—हर tool को अलग allow/revoke control, सीमित scope और logs।

डेटा मॉडल: User, PermissionGrant, ProviderProfile, ModelProfile, ComboProfile, Conversation, ScheduledTask, ToolAction, ToolApproval, PetSettings और ActivityEvent। शुरुआत में local DB (SQLite/Drift या अन्य supported Flutter solution); sync/accounts बाद में तभी जब सचमुच चाहिए।

बनाने का क्रम / milestone

1. **Stage 1 — Setup + MVP:** Flutter app, Home, fake voice UI, Tasks/Vision mock, Settings, real-device test।
2. **Stage 2 — Real conversation:** एक provider adapter, text chat, speech input/output, mic indicator, cost limits।
3. **Stage 3 — Smart tools:** reminders, app intents, scheduled tasks, approvals, activity logs।
4. **Stage 4 — Model Hub:** multiple providers, user keys, Combo, auto-failover, timeouts और budget।
5. **Stage 5 — 3D experience:** optimized avatar, animation controller, lip-sync, pet।
6. **Stage 6 — Advanced identity/vision:** camera scene awareness, optional identity matching, Owner/Guest, OS biometric gating।
7. **Stage 7 — Reliability / publishing:** automated tests, data policy, accessibility and Play policy audit, crash tracking, account deletion if accounts exist।

11. काम की चेकलिस्ट और फाइनल निर्णय

- ☐ Git और VS Code डाउनलोड और ओपन
- ☐ Flutter + Dart extension और Flutter SDK तैयार

- [ ] Android Studio + SDK tools + licenses तैयार
- [ ] Codex extension में ChatGPT Plus से लॉगिन
- [ ] असली फोन USB debugging से कनेक्ट
- [ ] `flutter doctor -v` के Android blockers दूर
- [ ] `flutter create manisha` + `flutter run` सफल
- [ ] Codex Prompt 1 से मनीषा का पहला UI
- [ ] `flutter analyze` + phone test + Git commit
- [ ] अगले चरण में voice + provider + approval mechanism को प्राथमिकता

> **एक लाइन की सलाह:** पहले VS Code + Codex + Flutter + Android Studio से असली Android फोन पर छोटा, स्थिर "मनीषा" ऐप बनाओ। उसी पर एक-एक करके voice, tools, provider router, 3D avatar, pet और advanced automation जोड़ो।

12. Official reference links (आगे की जाँच के लिए)

1. Flutter install: docs.flutter.dev/install/with-vs-code
2. Flutter Android setup: docs.flutter.dev/platform-integration/android/setup
3. Codex IDE: developers.openai.com/codex/ide
4. Codex plans: help.openai.com/en/articles/11369540-using-codex-with-your-chatgpt-plan
5. Agora RTC and Conversational AI pricing: agora.io/en/pricing
6. Google Play Accessibility policy: support.google.com/googleplay/android-developer/answer/10964491
7. Android scheduling: developer.android.com/develop/background-work/background-tasks/persistent
8. Android alarm policies: developer.android.com/develop/background-work/services/alarms
9. Android permissions: developer.android.com/training/permissions/requesting

नोट: सॉफ़्टवेयर और सेवाओं की कीमतें, free tiers, availability तथा Google Play/Android नीतियाँ बदल सकती हैं। वास्तविक रिलीज़ से पहले वर्तमान docs और वास्तविक डिवाइस पर व्यवहार दोबारा जाँचें। यह रोडमैप है, चालू ऐप या निश्चित डेवलपमेंट समय का वादा नहीं।